

JavaScript Roadmap — Table of Contents

Click any section below to jump directly to it.

Sections

- [Introduction](#)
- [Part I — JavaScript Fundamentals \(Week 1\)](#)
- [Part II — Advanced JavaScript Concepts \(Week 2\)](#)
- [Part III — JavaScript in the Browser \(Week 3\)](#)
- [Part IV — Modern JavaScript \(Week 4\)](#)
- [Part V — Final Project and Portfolio](#)

Introduction

(Your introduction content here)

Part I — JavaScript Fundamentals (Week 1)

(Part I content here) That's exactly how I'd do it. One thing I'd change, though: **I wouldn't make this just a roadmap—I would make it a real study handbook.** The goal is that every day you open it and simply follow the instructions, without having to decide what to study next.

Below is **Part 1** of the handbook.

Part I — Week 1 (Days 1–7)

Duration: July → August 17

Daily Study Time: 3 hours

Welcome!

This handbook was designed with one objective:

▮ **Become comfortable building applications with modern JavaScript before August 17.**

Unlike most tutorials, this roadmap doesn't try to teach every JavaScript feature.

Instead, it focuses on the concepts that professional developers use every day.

At the end of this month you should be able to build:

- Expense Tracker
- To-do List
- Weather App
- Movie Search App
- Notes App / Habit Tracker

without relying on tutorials.

Philosophy

Every day follows the same structure.

Theory (1 hour)
↓
Exercises (1h15min)
↓
Project (45min)

Every day you should finish by writing code.

Reading alone is **never enough**.

Resources

Primary

📖 javascript.info

<https://javascript.info/>

Secondary

📖 Eloquent JavaScript

<https://eloquentjavascript.net/>

Reference

📖 MDN

<https://developer.mozilla.org/>

Deep Dives

📖 You Don't Know JS Yet

<https://github.com/getify/You-Dont-Know-JS>

(Not required this week.)

Development Environment

Install:

- Visual Studio Code
- Node.js (latest LTS)
- Git
- GitHub account

Recommended extensions:

- ESLint
- Prettier
- JavaScript (ES6) snippets
- Error Lens
- Live Server

Folder Structure

Create one repository.

javascript-roadmap/

week1/

day1/

day2/

day3/

...

week2/

week3/

week4/

projects/

Commit every day.

Daily Routine

Activity	Time
Reading	60 min
Coding exercises	75 min
Project	45 min

Rules

- ✓ Type every example yourself.
- ✓ Never copy-paste.
- ✓ Push to GitHub every day.
- ✓ Google only after trying for 20 minutes.
- ✓ Read error messages.
- ✓ Experiment.

WEEK 1

Goal

Become fluent with JavaScript syntax.

Progress

Week Progress

□□□□□□□ 0%

DAY 1

Topics

- What is JavaScript?
 - Running JavaScript
 - Variables
 - let
 - const
 - Primitive Types
-

Reading

javascript.info

Read:

- JavaScript Fundamentals
 - Hello World
 - Code Structure
 - Variables
 - Data Types

Eloquent

- Chapter 1
-

Estimated reading

60 min

Coding

Create:

```
day1/
```

Exercises

1.

Declare variables

```
let age = 20;  
const PI = 3.14159;
```

2.

Swap two variables.

3.

Convert Celsius to Fahrenheit.

4.

Convert Fahrenheit to Celsius.

5.

Area of a circle.

6.

Average of 5 numbers.

7.

Simple calculator

(+ - * /)

Mini Challenge

Build

Age Calculator

Input

Birth year

Output

Current age

Python Comparison

Python

age = 20

JavaScript

let age = 20;

Difference:

JavaScript distinguishes mutable (`let`) and immutable (`const`) bindings.

Common Mistakes

❌ Using `var`

❌ Forgetting semicolons (even though they are optional)

❌ Reassigning a `const`

Notes

Daily Checklist

- ☐ Finished reading
- ☐ Finished exercises
- ☐ Built challenge
- ☐ Git commit

DAY 2

Topics

- Operators
- Strings
- Template Literals
- Boolean
- Comparison Operators
- Logical Operators

Reading

javascript.info

Read

- Operators
- Comparisons
- Conditional branching

Eloquent

Continue Chapter 1.

Exercises

Create

```
day2/
```

Build

- Number Guessing
- BMI Calculator
- Even or Odd
- Largest of three numbers
- Grade calculator

Mini Challenge

Password Validator

Requirements

Minimum

- 8 characters
- uppercase
- lowercase
- number

Python Comparison

Python

```
print(f"My age is {age}")
```

JavaScript

```
console.log(`My age is ${age}`);
```

Common Mistakes

Using

```
==
```

instead of

```
===
```

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge
- ☐ GitHub

DAY 3

Topics

- if
- else
- switch
- while
- do while
- for

Reading

javascript.info

Loops

Exercises

FizzBuzz

Multiplication Table

Prime Number

Factorial

Countdown

Mini Challenge

Rock Paper Scissors

Console only.

Python Comparison

Python

```
for i in range(10):
```

JavaScript

```
for(let i=0;i<10;i++)
```

Common Mistakes

Infinite loops

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Project
-

DAY 4

Topics

Functions

Parameters

Return

Arrow Functions

Reading

[javascript.info](#)

Functions

[Eloquent](#)

[Chapter 3](#)

Exercises

[Area functions](#)

[Temperature converter](#)

[Calculator](#)

[Prime checker](#)

[Palindrome](#)

Mini Challenge

[Calculator Library](#)

[Create functions](#)

```
add()
```

```
subtract()
```

```
multiply()
```

```
divide()
```

[Python Comparison](#)

[Python](#)


```
def add(a,b):
```

JavaScript

```
function add(a,b){  
}
```

Arrow

```
const add = (a,b)=>a+b;
```

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge

DAY 5

Topics

Arrays

Objects

Nested Objects

Reading

javascript.info

Objects

Arrays

Eloquent

Chapter 4

Exercises

Create

Student object

Book object

Movie object

Array of names

Average grades

Mini Challenge

Student Manager

Store

Name

Age

Grades

Average

Python Comparison

Python

```
student = {  
    "name": "John"  
}
```

JavaScript

```
const student = {  
    name: "John"  
}
```

Common Mistakes

Arrays are Objects.

Checklist

- ☐ Reading
- ☐ Exercises

DAY 6

Topics

Object Methods

Array Methods

for...of

for...in

Exercises

Reverse array

Largest number

Smallest number

Average

Remove duplicates

Mini Challenge

Library Manager

Books

Author

Pages

Read status

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 7

Review Day

No new theory.

Review

Variables

Functions

Arrays

Objects

Loops

Scope

Mini Project

Build

Console Expense Tracker

Features

Add expense

Delete expense

List expenses

Show total

Categories

(No GUI.)

Codewars

Solve

5 problems

Difficulty

8 kyu

7 kyu

Week Reflection

What concept was hardest?

What concept is still confusing?

What am I proud of this week?

Weekly Quiz

Can I answer YES?

Syntax

- ☐ Can I write a function without looking at notes?

Variables

- ☐ Do I know when to use let vs const?

Objects

- ☐ Can I create nested objects?

Arrays

- ☐ Can I manipulate arrays confidently?

Loops

- ☐ Can I solve problems using loops?

Functions

- ☐ Do I understand return values?

Git

- ☐ Did I commit every day?

Weekly Progress

 100%

Weekly Achievement

Unlock if completed:

- ☐ 🏆 JavaScript Beginner

Requirements

- Finish all readings
- Finish every exercise
- Complete Expense Tracker
- Push everything to GitHub

Looking Ahead

Week 2 will introduce the features that make JavaScript unique:

- Higher-order functions
- Closures
- Lexical scope
- Destructuring
- Spread operator
- Array methods (`map`, `filter`, `reduce`, etc.)
- `this`
- Building your first real application (Expense Tracker v2)

My recommendation

I genuinely think we can make this **much better**.

Right now this is about **15 pages**. By the end of the month, we'd have around **80–120 pages**.

I would also upgrade future parts by adding:

- **"Why?" sections** (explaining the reasoning behind each concept instead of just showing syntax).
- **ASCII diagrams** for concepts like scope chains, closures, memory references, the DOM tree, and the event loop.
- **"Interview Insight"** boxes highlighting questions commonly asked in JavaScript interviews.
- **XP (Experience Points)** for each completed day (gamifying the roadmap).
- **A mastery dashboard** tracking your confidence in each topic over time.
- **Reflection prompts** that encourage you to explain concepts in your own words (one of the most effective ways to retain knowledge).

By the end, it would feel less like a study guide and more like a professionally designed bootcamp handbook tailored specifically to



Part II — Advanced JavaScript Concepts (Week 2)

Part II — Week 2 (Days 8–14)

WEEK 2

Goal

This week you'll stop thinking of JavaScript as "just another language."

Instead, you'll learn the features that make JavaScript unique:

- References
- Higher-order functions
- Closures
- Lexical scope
- Destructuring
- Spread syntax
- Rest parameters
- Array methods
- `this`

By the end of the week you'll build your first **real application**.

Progress

00000000 0%

DAY 8

Today's Goal

Learn how JavaScript stores objects and arrays.

Topics

- Primitive vs Reference Types
- Object References
- Shallow Copy
- Deep Copy (concept)

Reading

javascript.info

- Object references and copying

Eloquent JavaScript

Review Chapter 4

Exercises

Create:

```
week2/day8/
```

Exercises

- Copy an object
- Modify copied object
- Observe original object
- Copy arrays
- Clone objects using spread

Mini Challenge

Create a **Contact Manager**

Features

- Add contacts
- Clone contacts
- Edit cloned contact
- Verify original object remains unchanged

Python Comparison

Python

```
b = a
```

JavaScript

```
const b = a;
```

Both variables point to the same object.

Common Mistakes

⚠ Thinking `=` creates a copy.

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge
- ☐ Git Commit

DAY 9

Today's Goal

Master modern object syntax.

Topics

- Destructuring
- Default values
- Nested destructuring
- Spread Operator
- Rest Parameters

Reading

javascript.info

- Destructuring Assignment
- Rest Parameters
- Spread Syntax

Exercises

- Swap variables
- Clone arrays
- Merge arrays
- Merge objects
- Function using rest parameters

Mini Challenge

Create a **Shopping Cart**

Features

- Merge carts
- Clone cart
- Calculate total

Common Mistakes

⚠ Confusing Spread (...) with Rest (...)

Checklist

- ☐ Reading
- ☐ Exercises

- ☐ Challenge

DAY 10

Today's Goal

Learn functional programming basics.

Topics

- Callback Functions
 - Higher-order Functions
-

Reading

javascript.info

- Advanced Working with Functions
-

Exercises

Create

- repeat()
- calculate()
- operate()

Pass functions as arguments.

Mini Challenge

Create your own

```
myMap()
```

without using `Array.map()`.

Interview Insight

Can you explain what a higher-order function is without giving examples?

If yes, you're understanding the concept.

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 11

Today's Goal

Master array methods.

Topics

- `map()`
 - `filter()`
 - `reduce()`
 - `find()`
 - `some()`
 - `every()`
 - `sort()`
-

Reading

javascript.info

Array methods

MDN

Read every method documentation.

Exercises

Given an array of users:

- Find adults
 - Sum ages
 - Sort alphabetically
 - Calculate average
 - Find oldest user
-

Mini Challenge

Create your own

```
Statistics Library
```

Functions

- `average()`
- `max()`
- `min()`
- `median()`

using array methods.

Python Comparison

Python

```
[x*x for x in nums]
```

JavaScript

```
nums.map(x => x*x)
```

Common Mistakes

☞ Using loops when map/filter/reduce are clearer.

Checklist

- ☐ Reading
 - ☐ Exercises
-

DAY 12

Today's Goal

Understand lexical scope and closures.

Topics

- Scope Chain
 - Lexical Scope
 - Closures
-

Reading

You Don't Know JS

- Get Started
 - Scope & Closures (selected chapters)
-

Exercises

Create

- Counter
 - Private Variable
 - Greeting Factory
-

Mini Challenge

Build

Counter Generator

Example

```
const counter = createCounter();

counter(); //1
counter(); //2
counter(); //3
```

Interview Insight

If you understand why the variable still exists after the function returns...

you understand closures.

Common Mistakes

⚠ Thinking closures copy variables.

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 13

Today's Goal

Understand `this`.

Topics

- `this`
 - Methods
 - Arrow Functions
 - Function Context
-

Reading

javascript.info

- Object Methods
 - `this`
-

Exercises

Create

Person object

Car object

Bank Account object

Print values using `this`.

Mini Challenge

Build

Bank Account

Methods

- `deposit()`
 - `withdraw()`
 - `balance()`
-

Common Mistakes

⚠ Arrow functions do **not** have their own `this` .

Python Comparison

Python

```
self
```

JavaScript

```
this
```

They're related, but not equivalent.

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 14

Review Day

No new theory.

Mini Project

Expense Tracker v1

Console application only.

Features

- Add Expense
 - Remove Expense
 - List Expenses
 - Categories
 - Monthly Total
 - Search Expenses
 - Statistics
-

Codewars

Solve

- 5 problems
 - 8 kyu
 - 7 kyu
-

Week Reflection

Biggest challenge

Most interesting concept

Concept I still need to review

Weekly Quiz

Can I answer YES?

References

- ☐ I understand object references.

Spread

- ☐ I know when to use spread.

Higher-order Functions

- ☐ I can pass functions as arguments.

Array Methods

- ☐ I know when to use `map()`.
- ☐ I know when to use `filter()`.
- ☐ I know when to use `reduce()`.

Closures

- ☐ I understand why closures work.

this

- ☐ I understand basic `this`.

Project

- ☐ Expense Tracker completed.

Git

- ☐ I committed every day.

Weekly Progress

 100%

Weekly Achievement

Unlock

- ☐ 📖 JavaScript Apprentice

Requirements

- Finish all readings
- Complete every exercise
- Complete Expense Tracker
- Solve 20+ coding problems

- Push everything to GitHub

Looking Ahead

Week 3 is where JavaScript becomes visual.

You'll learn:

- DOM
- HTML manipulation
- Events
- LocalStorage
- Fetch API
- Browser APIs

You'll also transform your console applications into real browser applications.

Part III — JavaScript in the Browser (Week 3)

Part III — Week 3 (Days 15–21)

JavaScript in the Browser

WEEK 3

Goal

This week you will move from writing isolated JavaScript programs to building **real browser applications**.

You will learn how JavaScript communicates with HTML and the browser environment.

By the end of this week you should understand:

- DOM manipulation
- Events
- Forms
- Dynamic interfaces
- LocalStorage
- JSON
- Fetch API
- Basic API consumption

You will transform your previous **Expense Tracker** from a console application into a real web application.

Progress

□□□□□□□□ 0%

DAY 15

The DOM (Document Object Model)

Today's Goal

Understand how JavaScript interacts with HTML.

Topics

- What is the DOM?
- DOM tree
- Selecting elements
- Changing content
- Changing styles
- Creating elements

Reading

javascript.info

Read:

- Document
- Searching: getElement*, querySelector

MDN

Review:

- Document API

Setup

Create:

```
week3/  
  
  expense-tracker/  
    index.html  
    style.css  
    script.js
```

Exercises

Create a page containing:

- A title
- A paragraph
- A button

Using JavaScript:

- Change the title
- Change the paragraph
- Change the button text

Mini Challenge

Create:

```
Dynamic Greeting
```

Requirements:

Input:

```
Your name
```

Output:

```
Hello, Mateus!
```

The message must appear without refreshing the page.

Python Comparison

Python:

```
print("Hello")
```

JavaScript in browser:

```
document.body.innerHTML = "Hello";
```

JavaScript can directly modify the webpage.

Common Mistakes

⚠ Changing the DOM before the page loads.

Solution:

```
DOMContentLoaded
```

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge
- ☐ Git Commit

DAY 16

DOM Manipulation

Today's Goal

Create and modify webpage elements dynamically.

Topics

- createElement()
- appendChild()
- remove()
- classList
- Attributes

Reading

javascript.info

Read:

- Modifying the document

Exercises

Create:

A list of tasks dynamically.

Requirements:

JavaScript creates:

```
<li>
```

elements.

Mini Challenge

Build:

```
Shopping List
```

Features:

- Add item
 - Remove item
 - Mark completed
-

Important Concept

The browser creates objects from HTML.

Example:

HTML:

```
<button>
Click
</button>
```

Becomes:

```
HTMLButtonElement
```

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 17

Events

Today's Goal

Understand how user interaction works.

Topics

- Events
- Event listeners
- Event objects
- Click events
- Keyboard events
- Forms

Reading

javascript.info

Read:

- Introduction to Events
- Browser Events

Exercises

Create:

- Counter button
- Dark mode button
- Keyboard detector

Mini Challenge

Build:

Calculator UI

Features:

- Buttons
- Input
- Operations

Event Flow



Common Mistakes

❌ Calling the function instead of passing it.

Wrong:

```
button.onclick = add();
```

Correct:

```
button.onclick = add;
```

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 18

Forms and Validation

Today's Goal

Learn how to handle user input correctly.

Topics

- Forms
 - Inputs
 - Submit events
 - Prevent default
 - Validation
-

Reading

javascript.info

Read:

- Forms
 - Form properties and methods
-

Exercises

Create:

Registration form.

Validate:

- Name
 - Email
 - Password
-

Mini Challenge

Create:

Fields:

- Description
- Value
- Category

Important Concept

Forms refresh the page by default.

Prevent:

```
event.preventDefault();
```

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge

DAY 19

LocalStorage and JSON

Today's Goal

Make data survive after closing the browser.

Topics

- LocalStorage
- JSON.stringify()
- JSON.parse()

Reading

MDN

Read:

- Web Storage API
- JSON

Exercises

Store:

- Username
- Theme preference
- List of tasks

Mini Challenge

Upgrade Expense Tracker.

Add:

- Save expenses
- Load expenses automatically

Important Concept

LocalStorage stores strings.

Example:

Object:

```
{
  name: "Book",
  price: 20
}
```

Needs conversion:

```
JSON.stringify(object)
```

Common Mistakes

⚠ Forgetting `JSON.parse` when retrieving data.

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge

DAY 20

Fetch API and External Data

Today's Goal

Learn how applications communicate with servers.

Topics

- HTTP basics
- APIs
- `fetch()`
- Promises review
- JSON responses

Reading

javascript.info

Read:

- Fetch

- Network Requests

Exercises

Consume:

Public APIs.

Examples:

- JSONPlaceholder
- Dog API
- Weather API

Mini Challenge

Build:

Random User Generator

Display:

- Name
- Picture
- Email

Request Flow

JavaScript

↓

fetch()

↓

API Server

↓

JSON Response

↓

Update DOM

Common Mistakes

⚠ Forgetting that fetch is asynchronous.

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 21

Project Day

Goal

Combine everything learned.

Expense Tracker v2

Upgrade your Week 2 project.

Requirements

Interface:

- HTML
- CSS
- JavaScript

Features:

Expenses

- Add expense
- Remove expense
- Edit expense

Categories

- Food
- Transport
- Entertainment
- Other

Storage

- Save automatically
- Load when opening

Statistics

Display:

- Total expenses
 - Number of expenses
 - Highest expense
-

Suggested Structure

```
expense-tracker/  
  
  index.html  
  
  css/  
    style.css  
  
  js/  
    app.js  
    storage.js  
    ui.js
```

Code Organization

Separate responsibilities.

Example:

```
storage.js
```

```
Handles:
```

- Saving
- Loading

```
ui.js
```

```
Handles:
```

- DOM updates

```
app.js
```

```
Handles:
```

- Application logic

Week Reflection

What was the hardest browser concept?

What DOM operation do I understand best?

What still feels confusing?

Weekly Quiz

DOM

- ☐ I understand what the DOM is.
- ☐ I can select HTML elements.
- ☐ I can create elements dynamically.

Events

- ☐ I understand event listeners.
- ☐ I can handle user input.

Forms

- ☐ I can validate forms.
- ☐ I understand preventDefault().

Storage

- ☐ I can save and retrieve data.
- ☐ I understand JSON conversion.

APIs

- ☐ I can consume an API.
- ☐ I understand asynchronous requests.

Project

- ☐ Expense Tracker v2 completed.

Git

- ☐ I committed my work.

Weekly Progress

 100%

Weekly Achievement

Unlock:

- ☐ 🏆 Browser Developer

Requirements:

- Complete Expense Tracker v2
- Use DOM manipulation
- Use LocalStorage
- Consume an API
- Push project to GitHub

Looking Ahead

Week 4 will focus on professional JavaScript:

- Modules
- Async/Await
- Error handling
- Classes
- Project architecture
- Final application development

You will start writing JavaScript closer to how it is written in real companies.

Part IV — Modern JavaScript (Week 4)

Part IV — Week 4 (Days 22–31)

Modern JavaScript and Professional Development

WEEK 4

Goal

This week you will transition from writing scripts and small applications to writing JavaScript with a more professional structure.

You will learn:

- ES Modules
- Code organization
- Promises deeply
- Async/Await
- Error handling
- Classes
- Prototypes (basic understanding)
- Application architecture

You will also build your third major project:

Movie Search Application

This project will combine everything learned so far.

Progress

□□□□□□□□ 0%

DAY 22

ES Modules

Today's Goal

Learn how professional JavaScript applications organize code.

Topics

- Modules
 - export
 - import
 - Default exports
 - Named exports
 - Module structure
-

Reading

javascript.info

Read:

- Modules, introduction
- Export and Import

MDN:

- JavaScript modules
-

Project Setup

Create:

```
movie-search/  
  
  index.html  
  
  css/  
    style.css  
  
  js/  
    main.js  
    api.js  
    ui.js  
    storage.js
```

Exercises

Create:

```
math.js
```

Export:

```
add()  
  
subtract()  
  
multiply()
```

Import them into another file.

Mini Challenge

Refactor Expense Tracker.

Separate:

```
data.js  
  
ui.js  
  
app.js
```

Important Concept

Before:

```
one huge file
```

After:

```
many small responsibilities
```

Common Mistakes

☒ Forgetting:

```
<script type="module">
```

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Refactor completed
-

DAY 23

Promises

Today's Goal

Understand asynchronous operations.

Topics

- Synchronous code
 - Asynchronous code
 - Promise states
 - resolve()
 - reject()
 - then()
 - catch()
-

Reading

javascript.info

Read:

- Promises
-

Exercise

Create:

```
const promise = new Promise();
```

Experiment with:

- resolve
 - reject
-

Mini Challenge

Create:

```
Fake Login System
```

Behavior:

Success:

Welcome user

Failure:

Invalid password

Mental Model

Promise:

Pending

↓

Success → fulfilled

or

Failure → rejected

Common Mistakes

⚠ Thinking asynchronous code runs immediately.

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge

DAY 24

Async/Await

Today's Goal

Write asynchronous code more naturally.

Topics

- async functions
- await
- try/catch
- Promise chaining

Reading

javascript.info

Read:

- Async/Await

Exercises

Convert:

Promise chains

into:

async functions.

Mini Challenge

Create:

```
Weather Fetcher
```

Flow:

1. User enters city.
 2. Fetch weather.
 3. Display result.
-

Comparison

Promise:

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data));
```

Async/Await:

```
const response = await fetch(url);
const data = await response.json();
```

Checklist

- ☐ Reading
 - ☐ Exercises
 - ☐ Challenge
-

DAY 25

Error Handling

Today's Goal

Learn how professional applications handle failures.

Topics

- try
- catch
- finally
- Custom errors

- API failures

Reading

javascript.info

Read:

- Error handling

Exercises

Handle errors from:

- Invalid API
- Invalid JSON
- Missing data

Mini Challenge

Improve Weather App.

Handle:

- Empty search
- Wrong city
- Server error
- Network failure

Professional Mindset

Bad:

Something broke.

Good:

User-friendly explanation.

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge

DAY 26

Classes and Object-Oriented JavaScript

Today's Goal

Understand JavaScript's class syntax.

Topics

- Classes
- Constructor
- Methods
- Inheritance
- Encapsulation

Reading

javascript.info

Read:

- Classes
- Class inheritance

Exercises

Create:

```
User class
```

```
BankAccount class
```

```
Product class
```

Mini Challenge

Create:

```
Movie class
```

Properties:

- title
- year
- rating

Methods:

- displayInfo()
- updateRating()

Important Concept

Classes are mostly syntax over prototypes.

You do not need advanced prototype knowledge yet.

Checklist

- ☐ Reading
- ☐ Exercises
- ☐ Challenge

DAY 27

JavaScript Under the Hood

Today's Goal

Understand what happens internally.

Topics

- Execution Context
 - Call Stack
 - Heap
 - Garbage Collection
 - Event Loop review
-

Resources

Watch:

"What the heck is the event loop anyway?"

by Philip Roberts

Exercise

Predict output:

```
console.log("A");

setTimeout(()=>{
  console.log("B");
},0);

console.log("C");
```

Expected:

```
A
C
B
```

Why?

JavaScript executes synchronous code first.

Checklist

- ☐ Watched video
 - ☐ Explained event loop
-

DAY 28

Movie Search App — Planning

Goal

Prepare final project architecture.

Project

Movie Search Application

Features

Basic:

- Search movies
- Display results
- Movie details

Intermediate:

- Favorites
- Pagination
- Loading animation
- Error handling

Advanced:

- Dark mode
 - LocalStorage
 - Filters
-

Architecture

```
movie-search/  
  
  js/  
  
    main.js  
  
    api.js  
  
    ui.js  
  
    storage.js  
  
    config.js
```

API

Use:

TMDB API

or another movie API.

Task

Create:

- HTML structure
- CSS layout
- API key setup
- Module structure

Checklist

- ☐ Project initialized
 - ☐ Files organized
 - ☐ API connected
-

DAY 29

Movie Search App Development

Goal

Implement core functionality.

Implement

API module:

```
api.js
```

Responsible for:

- Requests
- Data retrieval

UI module:

```
ui.js
```

Responsible for:

- Rendering movies
 - Updating DOM
-

Features

Implement:

- Search
 - Display results
 - Loading state
-

Checklist

- ☐ Search works
 - ☐ Movies display
 - ☐ Code organized
-

DAY 30

Movie Search App Improvements

Goal

Add professional features.

Add

- Favorites
 - LocalStorage
 - Error messages
 - Pagination
-

Code Review

Ask:

Can I explain:

- Every function?
 - Every module?
 - Every API call?
-

Checklist

- ☐ Favorites
 - ☐ Storage
 - ☐ Error handling
-

DAY 31

Final Polish

Goal

Prepare project for portfolio.

Improvements

Add:

- README.md
 - Screenshots
 - Instructions
 - Clean commits
-

GitHub Checklist

Repository contains:

```
README.md

index.html

css/

js/

screenshots/
```

Week Reflection

Most difficult concept?

What concept changed my understanding of JavaScript?

What should I review?

Weekly Quiz

Modules

- ☐ I know how import/export works.

Promises

- ☐ I understand Promise states.

Async

- ☐ I understand async/await.

Errors

- ☐ I can handle failures.

Classes

- ☐ I understand class syntax.

Architecture

- ☐ I can divide code into modules.

Projects

- ☐ Movie Search App completed.

GitHub

- ☐ Project published.
-

Weekly Progress

■■■■■■■■ 100%

Weekly Achievement

Unlock:

- ☐ 🏆 JavaScript Developer

Requirements:

- Complete Movie Search App
- Use modules
- Use async/await
- Consume an API
- Handle errors
- Publish project

Looking Ahead

The final stage is not learning new syntax.

It is learning how to combine everything.

Final days:

- Build one polished application.
- Improve previous projects.
- Review weak concepts.
- Prepare your portfolio.

You now have the foundation required to move toward:

- TypeScript
- React
- Node.js
- Full-stack JavaScript

Part V — Final Project and Portfolio

Part V — Final Phase (August 12–17)

Capstone Project, Review, and Portfolio Preparation

FINAL PHASE

Goal

This final phase is not about learning new JavaScript concepts.

The goal is to prove that you can combine everything you learned into a complete application.

You will:

- Build a polished project.
- Review weak concepts.
- Improve previous projects.
- Prepare your GitHub portfolio.
- Create a learning summary.

Progress

□□□□□□□□ 0%

Before Starting

You should already know:

JavaScript Fundamentals

- Variables
- Functions
- Arrays

- Objects
- Scope
- Closures

Modern JavaScript

- Destructuring
- Spread
- Modules
- Classes
- Async/Await

Browser

- DOM
- Events
- Forms
- LocalStorage
- Fetch API

Final Project Options

Choose ONE.

The objective is not complexity.

The objective is quality.

Option 1 — Habit Tracker 🌟 Recommended

Difficulty

☆☆☆☆ ☆

Features

Basic:

- Create habits
- Delete habits
- Mark daily completion

Intermediate:

- Streak counter
- Categories
- Progress statistics
- LocalStorage

Advanced:

- Calendar view
- Charts
- Dark mode

Concepts Practiced

- DOM
- Events
- LocalStorage
- Modules
- Array methods
- Application state

Option 2 — Kanban Board

Difficulty

★★★★

Features

Basic:

- Create tasks
- Delete tasks
- Edit tasks

Intermediate:

- Multiple columns
- Drag and drop
- Persistent storage

Advanced:

- Search
- Filters
- Animations

Concepts Practiced

- DOM manipulation
- Events
- Objects
- State management

Option 3 – Notes Application ☒

Difficulty

★★★★☆

Features

Basic:

- Create notes
- Delete notes
- Edit notes

Intermediate:

- Markdown preview
- Search
- Categories
- LocalStorage

Advanced:

- Dark mode
- Export notes
- Sorting

Concepts Practiced

- CRUD operations
- Storage
- Modules
- DOM
- Data organization

My Recommendation

Choose:

Notes Application

Why?

Because it represents real-world applications.

Almost every application contains:

- Data creation
- Data editing
- Data storage
- Searching
- Filtering
- User interaction

DAY 32

Project Planning

Goal

Design before coding.

Step 1 — Define Requirements

Write:

The application must allow users to:

Example:

The user can create, edit, and delete notes.

Step 2 — Design Data Structure

Example:

```
const note = {
  id: 1,
  title: "Study JavaScript",
  content: "Review closures",
  createdAt: Date.now()
};
```

Step 3 — Plan Modules

Example:

```
src/  
  
  main.js  
  
  storage.js  
  
  ui.js  
  
  app.js
```

Step 4 — Create Repository

README:

Include:

- Project description
- Features
- Technologies
- Screenshots
- Installation instructions

Checklist

- ☐ Requirements written
- ☐ Data structure designed
- ☐ Repository created
- ☐ Folder structure created

DAY 33

Building Core Features

Goal

Implement the minimum working version.

Development Order

Always follow:

```
Data  
  
↓  
  
Logic  
  
↓  
  
Interface  
  
↓  
  
Styling
```

Example:

Notes App

First:

```
createNote()  
deleteNote()  
updateNote()
```

Then:

Connect to buttons.

Checklist

- ☐ Create data model
 - ☐ Implement CRUD
 - ☐ Connect JavaScript to HTML
-

DAY 34

Adding Persistence

Goal

Make the application remember information.

Implement

LocalStorage:

Save:

```
localStorage.setItem()
```

Retrieve:

```
localStorage.getItem()
```

Requirements

When the user:

- Refreshes page
- Closes browser
- Opens again

Data remains.

Checklist

- ☐ Storage works
 - ☐ Data loads automatically
 - ☐ No duplicated data
-

DAY 35

Improving User Experience

Goal

Make the application pleasant to use.

Add

- Loading states
- Error messages
- Empty states
- Better layout
- Responsive design

Example

Bad:

Error

Good:

Unable to save note.
Please try again.

Checklist

- ☐ User feedback added
- ☐ Errors handled
- ☐ Interface improved

DAY 36

Code Quality Day

Goal

Improve your code like a professional developer.

Refactoring Checklist

Ask:

Naming

Can variables explain themselves?

Bad:

```
let x;
```

Good:

```
let totalExpenses;
```

Functions

Does each function do one thing?

Bad:

```
function handleEverything()
```

Good:

```
saveNote()
```

```
renderNotes()
```

```
deleteNote()
```

Organization

Are files separated logically?

Checklist

- ☐ Removed duplicated code
- ☐ Improved names
- ☐ Added comments where necessary
- ☐ Organized files

DAY 37

Portfolio Preparation

Goal

Transform code into a professional project.

GitHub Checklist

Repository:

```
project-name/
```

```
README.md
```

```
index.html
```

```
css/
```

```
js/
```

```
assets/
```

README Structure

```
# Project Name
```

```
## Description
```

```
What does this application do?
```

```
## Features
```

- Feature 1
- Feature 2

```
## Technologies
```

- HTML
- CSS
- JavaScript

```
## Screenshots
```

```
(images)
```

```
## Installation
```

```
Instructions
```

```
## Future Improvements
```

```
Ideas
```

Git Practices

Good commits:

```
Add notes creation feature
```

```
Implement local storage
```

```
Fix search filtering bug
```

Avoid:

```
update  
changes  
final
```

Checklist

- ☐ README created
 - ☐ Screenshots added
 - ☐ Repository organized
 - ☐ Commits cleaned
-

DAY 38

Final Review

Goal

Identify weaknesses.

Review Topics

JavaScript Core

Can I explain:

- ☐ let vs const
- ☐ Scope
- ☐ Closures
- ☐ Objects
- ☐ Array methods

Browser

Can I explain:

- ☐ DOM
- ☐ Events
- ☐ LocalStorage

Async

Can I explain:

- ☐ Promises
- ☐ Async/Await
- ☐ Fetch

Architecture

Can I explain:

- ☐ Modules
 - ☐ Separation of responsibilities
-

DAY 39

Final Reflection

Write a Learning Report

Answer:

1. What changed in my understanding of JavaScript?

2. What concept was hardest?

3. What can I build now?

4. What will I learn next?

Suggested:

- TypeScript
 - React
 - Node.js
 - Testing
 - Databases
-

Final Achievement

🏆 JavaScript Foundation Complete

Requirements:

- ☐ Completed all weeks
 - ☐ Built at least 5 projects
 - ☐ Published GitHub repositories
 - ☐ Understand core JavaScript concepts
 - ☐ Can build applications independently
-

Complete Roadmap Summary

Week 1
JavaScript Fundamentals
↓
Week 2
Advanced Language Features
↓
Week 3
Browser Development
↓
Week 4
Professional JavaScript
↓
Final Phase
Portfolio Project

After August 17

Recommended next path:

Month 2

TypeScript

Learn:

- Types
- Interfaces
- Generics
- Advanced tooling

Month 3

React

Learn:

- Components
- Props
- State
- Hooks
- Routing

Month 4

Node.js

Learn:

- Express
- APIs
- Databases
- Authentication

Final Advice

Do not measure success by how many tutorials you completed.

Measure success by:

- How independently you solve problems.
- How well you understand your own code.
- How confidently you can modify existing applications.

The objective of this roadmap was not to memorize JavaScript.

The objective was to become someone who can **use JavaScript as a tool to create things**.